

TaxonVision-MLOps

End-to-End Architectural and Operational Strategy Report

Systems Engineering

Benjamin Förster

October 2, 2026

Abstract

TaxonVision-MLOps establishes an automated, production-grade Machine Learning Operations platform for fine-grained biological species identification. Transitioning away from ad-hoc, notebook-bound model development, this architecture utilises a unified open-source toolchain (DagsHub, DVC, MLflow, Optuna) cleanly mapped to a hybrid compute topology (GitHub Actions and local bare-metal GPU runners). In order to comply strictly with ethical and regulatory standards, the system firmly enforces epistemic safety via Split Conformal Prediction and Energy-Based Out-of-Distribution (OOD) detection, thereby ensuring mathematical coverage guarantees in production. This document carefully outlines the end-to-end pipeline, testing strategies, and core architectural trade-offs driving the system's overall design.

Contents

Abstract	i
1 Architectural Context and Operational Maturity	1
1.1 Data Topography and Open-Access Mandates	1
1.2 The Operational Bottleneck in Biodiversity Monitoring	1
1.3 The MLOps Evolution Paradigm	2
2 Physical Infrastructure and Data Topology	3
2.1 Heterogeneous Silicon and Thermal Determinism	3
2.2 Cryptographic Storage Economics	4
2.2.1 Circumventing the 50 TB Trap	4
2.2.2 Bounded Exemplar Storage Mathematics	4
2.2.3 External Remote Fallback Topology	4
3 The Open-Source MLOps Toolchain	5
3.1 Content-Addressable Storage and Cryptographic Versioning	5
3.2 The Unified Management Plane	5
3.3 Bayesian Hyperparameter Optimisation and Telemetry	6
3.3.1 Intelligent Search Mechanics	6
3.3.2 The MLflow Interlock Architecture	7
4 Dual Pipeline Architecture	8
4.1 Asymmetric Iteration and Execution Decoupling	8
4.2 Automated CI/CD Orchestration and Registry Promotion	8
4.3 Convergence at the Serving Boundary	9
5 Vision RAG & Multimodal Knowledge Graph	11
5.1 Ingestion Engine & Reference Exemplar Catalogue Curation	11
5.2 Visual Entity Linking: Parent-Child Chunking	12
5.3 Vector Database: Single-Stage Filtered HNSW	12
5.4 Conformal-Driven Lazy Execution	13
5.5 Multi-Modal Re-Ranking via Reciprocal Rank Fusion	14
5.6 Multimodal Context Synthesis & VLM Diagnostics	14
5.7 Closed-Loop Active Learning & Taxonomic Mutations	15
5.8 End-to-End Latency & Resource Allocation	15
6 Epistemic Safety & Active Learning	16
6.1 Energy-Based Out-of-Distribution Detection	16
6.2 Spatial-Temporal Constraint Masking	17
6.3 Distribution-Free Uncertainty via Split Conformal Prediction	18
6.3.1 Calibration Protocol and Quantile Estimation	18
6.3.2 Prediction Set Construction & Marginal Coverage Guarantee	18
6.4 Conformal Triage Routing & Active Learning Feedback	19
6.4.1 Active Learning Query Strategies	19

7	Observability, Testing & Deployment	21
7.1	Containerisation and Multi-Stage Runtimes	21
7.1.1	Multi-Stage Image Compilation	21
7.1.2	Kernel-Level Process Isolation	22
7.2	Kubernetes Cluster Orchestration and Workload Routing	22
7.2.1	Asymmetric Pod Architecture and Ingress Routing	22
7.2.2	Self-Healing and Health Probing	22
7.2.3	Horizontal Pod Autoscaling on Custom Telemetry	23
7.3	Telemetry, Metric Exporters, and Concept Drift Monitoring	23
7.3.1	Prometheus Telemetry Instrumentation	23
7.3.2	Mathematical Formulation of Feature Drift via Maximum Mean Discrepancy	24
7.3.3	Automated Alerting and Continuous Retraining Triggers	24
7.4	Multi-Tier Verification and Testing Strategy	25
7.4.1	Tier 1: Upstream API Mocking	25
7.4.2	Tier 2: Object Storage and Pipeline Mocking	25
7.4.3	Tier 3: Mathematical Invariant Testing	25
7.4.4	Tier 4: Statistical Metric Selection and Anomaly Verification	26
8	Hardware Optimisation and Memory Mathematics	28
8.1	Dynamic Quantisation and Tensor Compression	28
8.2	Marginal Coverage Guarantees in Conformal Prediction	29
8.3	Memory Complexity of Hierarchical Vector Graphs	29

Chapter 1

Architectural Context and Operational Maturity

The transition from localised machine learning prototypes to global ecological monitoring platforms requires us to fundamentally rethink how our systems handle scale, uncertainty, and regulatory compliance. In order to achieve this, this chapter establishes the baseline operational context for the TaxonVision-MLOps architecture. We first define the scale of citizen-science data streams, then identify the failure modes of manual data science workflows, and finally formalise the phased Machine Learning Operations (MLOps) maturity framework which is required to sustain a robust production environment.

1.1 Data Topography and Open-Access Mandates

Global biodiversity portals such as iNaturalist and the Global Biodiversity Information Facility (GBIF) aggregate continuous streams of species observations worldwide. Combined, these ecosystems already host approximately 270 million raw observations, of which roughly 170 million have achieved “research grade” status through rigorous community validation. We must acknowledge that processing this high-volume stream requires strict cryptographic and legal compliance safeguards right at the ingestion edge.

In order to enforce open-access mandates, the ingestion engine inspects incoming observation metadata headers prior to staging objects locally. A cryptographic licence filtering module evaluates record metadata against allowed open-access definitions, thereby enforcing strict inclusion filters for Creative Commons Zero (CC0), Creative Commons Attribution (CC-BY), and Creative Commons Attribution-NonCommercial (CC-BY-NC). Any incoming observation tagged with restrictive or commercial licences (for example, CC-BY-ND or CC-BY-SA restricted variants) is explicitly dropped at the network boundary. Furthermore, to comply with GBIF and iNaturalist scientific lineage requirements, an automated attribution preservation mechanism extracts observation GUIDs, contributor ORCIDs, and original licence headers, serialising them alongside downstream feature vectors in order to maintain absolute lineage traceability.

1.2 The Operational Bottleneck in Biodiversity Monitoring

Fine-grained biological species identification at a global scale represents a formidable computational challenge. Citizen-science platforms generate continuous, high-volume streams of unstructured visual observations across highly variable environments. Historically, translating these raw field photographs into verified ecological intelligence has been severely constrained by fragile, ad-hoc machine learning engineering.

When predictive models are trained in isolated Jupyter notebooks – a state defined as Level 0 (Manual Process) maturity – they already suffer from non-reproducible software environments, untracked parameter drift, and silent degradation under natural distribution shifts. In an operational setting, these isolated scripts break down entirely during the handoff between data science and deployment teams. This renders the system incapable of sustaining continuous updates or scaling to support thousands of distinct taxa.

1.3 The MLOps Evolution Paradigm

In order to reliably process citizen-science streams, the TaxonVision-MLOps architecture forces a paradigm shift. We must treat visual identification not as a static model, but as a continuously evolving enterprise service. This requires abandoning manual intervention in favour of a strictly automated, tiered Machine Learning Operations (MLOps) framework designed to advance directly to Level 3 and Level 4 maturity.

By framing the infrastructure as an automated pipeline, the architecture mandates three foundational pillars:

1. **Continuous Integration and Deployment (CI/CD):** The implementation of rapid, automated pipeline code releases and testing triggered by version control events.
2. **Cryptographic Reproducibility:** The mathematically verifiable versioning of both the execution logic (via Git) and the underlying data states (via Data Version Control).
3. **Epistemic Safety:** The rejection of raw, uncalibrated softmax probability scores in favour of Split Conformal Prediction. This provides formal, distribution-free mathematical guarantees that the true species is contained within the prediction set up to a user-defined error tolerance, α .

Table 1.1: TaxonVision-MLOps Maturity Framework

Maturity Level	Infrastructure Configuration	Operational Focus
0: Manual	Isolated interactive notebooks; manual state handoffs.	Localized algorithm prototyping.
1: Pipeline	Automated Directed Acyclic Graphs (DAGs); basic data validation.	Establishing a continuous training baseline.
2: CI/CD	GitHub Actions integration; dynamic ONNX graph export.	Rapid, repeatable code releases.
3: Feedback	Automated telemetry scraping; Active Learning triage queues.	Data-engine iterations and real-time concept drift detection.
4: Safety	Split Conformal Prediction; Energy-Based OOD rejection.	Distribution-free coverage guarantees $(1 - \alpha)$.

Chapter 2

Physical Infrastructure and Data Topology

The computational demands of high-throughput vision architectures strictly dictate physical and operational determinism. In order to move beyond volatile cloud abstractions, this chapter establishes the bare-metal hardware topography, memory budgeting, and cryptographic data storage strategies which are necessary to sustain continuous model training and real-time inference without performance degradation or unnecessary cloud expenditures.

2.1 Heterogeneous Silicon and Thermal Determinism

Training and deploying dense Vision Foundation Models (VFMs) at high throughput necessitate deterministic physical execution. Relying on shared public cloud infrastructure inevitably introduces hypervisor virtualisation overhead and variable PCIe bandwidth. This obscures true performance metrics during latency profiling. Consequently, the platform utilises a dedicated bare-metal host architecture.

The computational core relies on an Intel Core i7-14700K processor, thereby leveraging its heterogeneous core topology to explicitly isolate workloads. High-frequency neural execution paths and PyTorch gradient calculations are pinned strictly to the Performance-cores (P-cores), preserving the L2 and L3 cache lines for the mathematical hot loops. Conversely, background asynchronous I/O operations – such as streaming HTTP requests, dataset extraction, and database indexing – are relegated to the Efficiency-cores (E-cores) in order to prevent CPU context switching.

Neural network acceleration is driven by an NVIDIA RTX 5070 Ti possessing 16 GB of GDDR7 VRAM. This provides the necessary computational throughput and absolute physical stability required to reliably prevent clock-speed variance during critical CI/CD latency validation tests and continuous multi-day benchmarking suites.

Storage I/O is managed across a 2 TB NVMe SSD secured via LUKS block-level encryption. In order to maximise read bandwidth across the PCIe bus when streaming millions of small image chunks, the underlying BTRFS filesystem utilises transparent `zstd:3` compression. However, we must consider that BTRFS is a Copy-on-Write (CoW) filesystem. This fundamentally conflicts with continuous database writes and Data Version Control (DVC) chunk caching, leading to severe disk fragmentation. In order to resolve this, a dedicated storage subvolume is initialised with the NOCOW attribute (`chattr +C`), ensuring contiguous physical block allocations for all local data caches.

2.2 Cryptographic Storage Economics

A primary architectural constraint for open-source and low-budget MLOps is data storage costs. The platform standardises on DagsHub as the central management plane, which enforces a 20 GB free-tier storage limit on its DVC remote. We find that this limit is entirely viable, provided that the pipeline enforces a strict physical separation between bulk raw observation streams and versioned metadata artefacts.

2.2.1 Circumventing the 50 TB Trap

The raw citizen-science dataset hosted on the AWS Open Data registry already exceeds 50 Terabytes. Mirroring this volume locally or to a Git-tracked remote would be financially and computationally catastrophic. The system explicitly avoids this through zero-duplication streaming. Our ingestion scripts stream the massive TAR archives directly from AWS Open Data into memory, leveraging zero-copy Apache Arrow buffers.

Rather than storing the raw images, DVC tracks the cryptographically validated Parquet index manifests. These lightweight tabular files contain the observation IDs, DarwinCore taxonomic metadata, source URLs, and SHA-256 validation hashes. A Parquet manifest indexing 10 million distinct biological observations typically consumes less than 300 MB of disk space.

2.2.2 Bounded Exemplar Storage Mathematics

For the actual visual data, the Vision RAG architecture curates a bounded Reference Exemplar Catalogue, strictly capped at 50 validated environmental crops per species.

For a production deployment spanning 2,000 baseline taxa, this yields exactly 100,000 curated images. Assuming a compressed WebP format averaging 60 KB per crop, the mathematical storage footprint evaluates as follows:

$$\text{Storage}_{\text{Exemplars}} = 100,000 \text{ images} \times 60 \text{ KB} = 6,000,000 \text{ KB} \approx 6.0 \text{ GB}$$

Furthermore, dynamic INT8 quantised ONNX model checkpoints range from 20 MB to 350 MB each. Retaining a rolling buffer of the most recent production release candidates consumes well under 2.0 GB.

$$\begin{aligned} \text{Storage}_{\text{Total}} &= 300 \text{ MB (Manifests)} + 6.0 \text{ GB (Exemplars)} \\ &\quad + 2.0 \text{ GB (ONNX Models)} + 1.5 \text{ GB (DVC Churn)} \approx 9.8 \text{ GB} \end{aligned}$$

This peak allocation naturally consumes less than 50% of the DagsHub free-tier allowance, thereby leaving significant overhead for active learning annotations.

2.2.3 External Remote Fallback Topology

As the catalogue scales towards 10,000 global taxa, the exemplar storage requirement will linearly increase to approximately 30 GB, thereby breaching the native quota. In this scenario, the DVC remote backend is simply retargeted to an external, zero-egress S3-compatible bucket (such as Cloudflare R2 or a localised MinIO cluster). DagsHub then continues to render the Directed Acyclic Graphs (DAGs) and track Git-to-DVC commit diffs, successfully decoupling the management interface from the physical storage layer.

Chapter 3

The Open-Source MLOps Toolchain

In order to construct a production-grade machine learning platform without incurring prohibitive enterprise cloud licensing costs, our architecture integrates a curated selection of open-source components. However, merely assembling disparate tools is insufficient. True operational resilience requires a deep, mechanical interlocking between data versioning, experiment telemetry, and hyperparameter optimisation.

3.1 Content-Addressable Storage and Cryptographic Versioning

Standard version control systems, such as Git, are engineered to track line-by-line code modifications within directed acyclic graphs of text files. Applying this paradigm to binary biological datasets is architecturally catastrophic. Committing a 50-gigabyte archive of high-resolution images forces the repository database to duplicate entire assets upon the slightest modification. This inevitably leads to memory exhaustion and repository collapse.

The system safely circumvents this limitation by utilising Data Version Control (DVC), which implements content-addressable storage. Rather than tracking the physical image matrices, DVC computes a unique SHA-256 cryptographic hash based strictly on the pixel content of the target directory. The heavy binary payloads are routed directly to a remote storage bucket, while a lightweight pointer file (for example, `dataset.dvc`) containing only the resultant hash and file metadata is generated.

This pointer file is subsequently committed to Git. When a deployment runner or data scientist executes a checkout command for a specific historical commit, the DVC client reads the hash from the pointer file, resolves the asset location in the remote cache, and constructs near-instantaneous filesystem reflinks (or hardlinks) into the local workspace. This dual-layered approach reliably decouples the heavy binary storage from the codebase, whilst guaranteeing mathematically verifiable, bit-for-bit data reproducibility.

3.2 The Unified Management Plane

Operating a distributed MLOps toolchain traditionally requires provisioning disparate infrastructure: an AWS S3 bucket for data object storage, a PostgreSQL database for experiment telemetry, and a complex matrix of Identity and Access Management (IAM) policies in order to broker communication between them.

In order to eliminate this DevOps overhead, the architecture standardises on DagsHub as the central management plane. By mirroring the primary GitHub repository, DagsHub acts as an automated nexus that instantly provisions a secure, S3-compatible remote storage backend for DVC and a managed MLflow Tracking Server. This integration effectively collapses the required infrastructure footprint, allowing self-hosted execution runners and local development environments to authenticate and stream both multi-gigabyte datasets and real-time loss metrics through a unified API token gateway.

3.3 Bayesian Hyperparameter Optimisation and Telemetry

Model convergence, latency minimisation, and epistemic reliability depend fundamentally on tuning discrete and continuous training hyperparameters. Rather than viewing hyperparameter configuration and experiment tracking as disjoint manual procedures, the architecture integrates Bayesian optimisation algorithms directly with distributed metric logging pipelines in order to form a self-contained optimisation loop.

3.3.1 Intelligent Search Mechanics

Neural network architectures require the optimisation of external hyperparameters, such as learning rates, weight decay penalties, and conformal confidence thresholds. Traditional Grid Search algorithms evaluate these spaces exhaustively, incurring a computational cost of $\mathcal{O}(S^D)$, where S represents the number of discrete steps evaluated across D hyperparameter dimensions. On a single-node bare-metal architecture, this geometric explosion renders hyperparameter tuning computationally unviable.

The platform replaces brute-force evaluation with Optuna, a framework driven by the Tree-structured Parzen Estimator (TPE). TPE abandons blind search grids in favour of Bayesian probability. Instead of modelling the objective function directly, it models the parameter probability distributions conditioned on historical trial performance. Formally, for a given hyperparameter configuration θ and a target loss metric y , TPE models the conditional probability $P(\theta \mid y)$ as follows:

$$P(\theta \mid y) = \begin{cases} \ell(\theta) & \text{if } y < y^* \\ g(\theta) & \text{if } y \geq y^* \end{cases}$$

In this formulation:

- θ is the specific hyperparameter configuration (for example, learning rate) being evaluated.
- y is the empirical loss achieved by executing the network with configuration θ .
- y^* is a dynamically computed scalar quantile threshold separating high-performing trials from poor-performing trials.
- $\ell(\theta)$ is the probability density function formed by the parameters of successful past trials ($y < y^*$).
- $g(\theta)$ is the probability density function formed by the parameters of unsuccessful past trials ($y \geq y^*$).

The algorithm iteratively selects new configurations that maximise the ratio $\ell(\theta)/g(\theta)$. This guides the GPU to sample from regions mathematically correlated with low loss, whilst preserving enough variance to escape local minima.

In order to further conserve hardware cycles, Optuna executes the Asynchronous Successive Halving Algorithm (ASHA). If a trial's validation metrics fall into the bottom quartile during its initial training epochs, ASHA aggressively prunes (terminates) the run. This immediately frees the Tensor Cores to evaluate a more promising branch of the probability distribution.

3.3.2 The MLflow Interlock Architecture

While Optuna efficiently orchestrates the search mathematics locally, it inherently lacks the infrastructure to permanently archive model provenance across a distributed engineering team. MLflow bridges this gap, operating as the continuous flight data recorder for our training pipeline.

The two systems are strictly coupled via an `MLflowCallback` hook. As the pipeline executes, the overarching Optuna *Study* is registered as a root experiment in MLflow. Every individual parameter combination evaluated by Optuna is recorded as a nested child run. During these nested executions, MLflow streams the step-by-step metrics (for example, training loss, top-1 accuracy) and the sampled parameters (for example, θ) directly to the DagsHub dashboard. Upon completion, physical artefacts – such as the serialised ONNX graphs and validation histograms – are uploaded to the artefact registry. Programmatic triggers then evaluate the Pareto frontier of all logged trials, automatically promoting the optimal model checkpoint to the MLflow Model Registry, staging it for formal CI/CD deployment testing.

Chapter 4

Dual Pipeline Architecture

A common anti-pattern in machine learning deployments is the monolithic pipeline, where neural network training sequences and data indexing tasks are tightly coupled into a single execution graph. The TaxonVision-MLOps architecture explicitly rejects this model. Instead, it establishes a dual-pipeline topology that decouples the weight-centric classification engine from the data-centric Multimodal Knowledge Graph (MMKG). These pipelines maintain independent lifecycles, converging exclusively at the inference serving boundary.

4.1 Asymmetric Iteration and Execution Decoupling

The necessity for architectural decoupling stems directly from the fundamentally asymmetric nature of biological modelling operations.

Pipeline 1: The Classification Engine. The primary neural backbone (for example, an optimised BioCLIP-2 or DINOv3 transformer) is computationally expensive to train but relatively static in production. Refining its weights requires intensive GPU Tensor Core utilisation, distributed gradient synchronisation, and dynamic ONNX quantisation calibration. However, once an optimal checkpoint is compiled, the model architecture may remain unchanged for weeks or months.

Pipeline 2: The Taxonomic MMKG. Conversely, the underlying citizen-science data stream is highly volatile. The pipeline must ingest continuous drops of new iNaturalist research-grade observations. More critically, biological taxonomies are frequently subjected to scientific realignments – species are split into novel taxonomic branches or lumped together under unified nomenclature. Pipeline 2 is an I/O-bound vector-indexing workflow responsible for parsing DarwinCore Parquet manifests, extracting zero-shot organism crops, filtering open-source licences, and appending dense embeddings to the Hierarchical Navigable Small World (HNSW) graph.

Forcing Pipeline 1 to re-execute a heavy neural network training sweep merely because Pipeline 2 updated an exemplar image for a specific beetle species wastes immense computational resources. Furthermore, physically separating these domains enforces strict failure isolation. If a malformed DarwinCore archive from an upstream provider induces a schema violation, Pipeline 2 will halt securely. However, Pipeline 1 remains entirely unimpeded, preserving our ability to compile, test, and deploy critical hot-fixes to the classification logic.

4.2 Automated CI/CD Orchestration and Registry Promotion

In order to transition from manual experimentation to a Level 3 MLOps deployment, the independent pipelines must be strictly governed by Continuous Integration and Continuous Deployment (CI/CD) triggers.

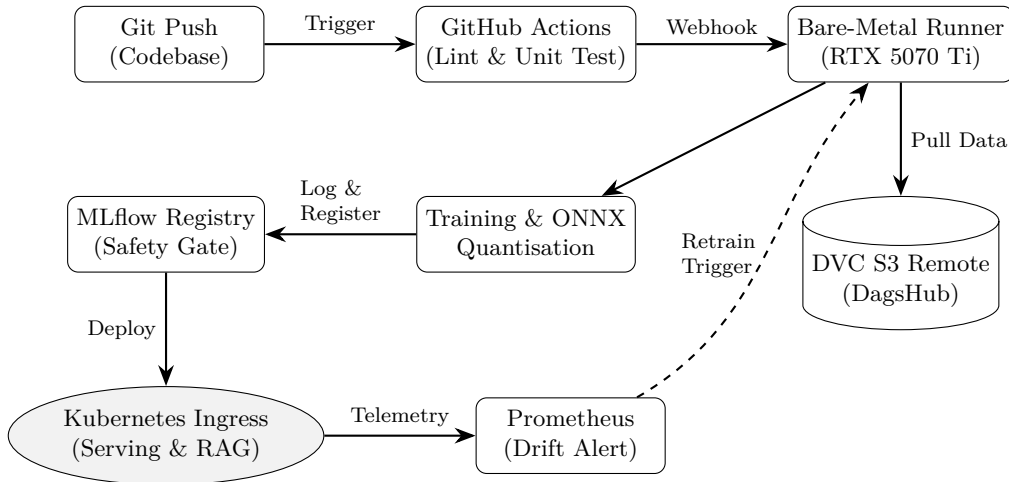


Figure 4.1: Dual-Pipeline CI/CD Architecture Flow mapping code and data states to production Kubernetes deployments.

When engineers push codebase modifications to the central repository, GitHub Actions cloud runners immediately execute lightweight static analysis and logical unit tests. Upon passing, a secure webhook triggers the self-hosted, bare-metal GPU runner to initiate the heavy execution graph. In order to authenticate with the tracking infrastructure, the GitHub Actions runner injects encrypted repository secrets (for example, `MLFLOW_TRACKING_URI`) directly into the local execution environment.

As the training pipeline initialises, it binds the MLflow telemetry directly to the exact Git commit SHA that triggered the run, thereby ensuring absolute cryptographic provenance between the source code and the resulting model weights:

```
1 mlflow.set_tag("git.commit", os.environ.get("GITHUB_SHA"))
```

Crucially, the CI/CD pipeline enforces an automated epistemic safety gate prior to deployment. Before any compiled model is allowed to enter the registry, a standalone invariant test loads the candidate ONNX graph and evaluates it against a holdout calibration dataset. The script formally asserts that the empirical marginal coverage of the Split Conformal Prediction set, $|C|$, meets or exceeds the mathematical bound $1 - \alpha$ (for example, 95% coverage, where $\alpha = 0.05$). If the empirical coverage falls below this strict threshold, the pipeline throws a fatal assertion error, blocking the Git merge and effectively preventing the uncalibrated model from reaching production.

If the safety gate is cleared, the system programmatically interacts with the MLflow API to transition the newly validated artefact from a staging environment directly into production readiness:

```
1 client.transition_model_version_stage(
2     name="TaxonVision-Backbone",
3     version=mv.version,
4     stage="Production"
5 )
```

4.3 Convergence at the Serving Boundary

The two distinct pipelines exist in absolute isolation until the precise moment of user inference. The FastAPI serving application acts as the convergence node, simultaneously loading the INT8

quantised ONNX graph from Pipeline 1 into memory, whilst mounting the live IVF-HNSW vector index compiled by Pipeline 2.

When a field photograph is submitted to the API, it is passed through the fast-path ONNX backbone. If the epistemic safety layer determines the classification is decisive – meaning the conformal prediction set yields exactly one candidate ($|C| = 1$) – the pipeline returns the prediction immediately, bypassing the vector database entirely. If, however, the model exhibits severe ambiguity ($|C| > 1$), the serving application lazy-loads Pipeline 2’s HNSW index, executes a single-stage filtered traversal, and retrieves the top- k reference exemplar crops in order to synthesise a robust, comparative visual context.

Chapter 5

Vision RAG & Multimodal Knowledge Graph

Standard text-centric Retrieval-Augmented Generation (RAG) paradigms typically rely on document chunking, token embedding, and semantic similarity search in order to ground large language models. In visual ecology, we must fundamentally transform this concept into multimodal equivalents. Citizen-science platforms aggregate massive volumes of environmental imagery, but we must acknowledge that raw field photographs are highly dense, noisy, and unstructured. Rather than attempting to compute and search vector embeddings across the entire 170-million-image iNaturalist corpus, the TaxonVision-MLOps architecture couples a curated Reference Exemplar Catalogue with single-stage spatial-temporal filtered graph traversal and conformal-driven lazy execution.

5.1 Ingestion Engine & Reference Exemplar Catalogue Curation

A naive approach to visual retrieval attempts to pass every available field photograph through a deep Vision Foundation Model (VFM) in order to construct an exhaustive vector database. In the context of global biodiversity monitoring, this brute-force strategy requires generating dense embeddings for more than 170 million observations. Storing 512-dimensional 32-bit floating-point (FP32) representations for this volume would require over 340 GB of addressable memory purely to hold the vector matrices in RAM, alongside years of continuous GPU compute on workstation-grade silicon.

The platform actively circumvents this computational barrier by curating a deterministic Reference Exemplar Catalogue. Instead of tracking all historical user submissions, our ingestion pipeline filters raw DarwinCore observation records according to four strict quality predicates:

1. **Validation Tier:** Observations must hold official `research_grade` status, confirming agreement across three or more independent community experts.
2. **Licence Compliance:** Records must possess open-access metadata headers matching Creative Commons Zero (CC0), Creative Commons Attribution (CC-BY), or Attribution-NonCommercial (CC-BY-NC). Commercial or share-alike variants (CC-BY-ND, CC-BY-SA) are explicitly dropped at the ingestion boundary.
3. **Signal Quality:** Raw photographs must exceed a minimum resolution of 1024×1024 pixels. Furthermore, upstream zero-shot object detection must isolate the primary organism with a detection confidence score greater than 0.85.
4. **Diversity Sampling:** The catalogue selects up to 50 exemplar observations per taxon.

In order to capture seasonal and geographical morphological variations, observations are partitioned into discrete spatial clusters using Uber H3 hexagonal spatial cells at Resolution 4.

By bounding the catalogue to 2,000 baseline taxa with 50 exemplars each, the vector database maintains an upper bound of 100,000 reference vectors. The total memory required to hold these 512-dimensional FP32 vectors in RAM evaluates to:

$$\text{Memory}_{\text{Vectors}} = 100,000 \times 512 \times 4 \text{ bytes} \approx 204.8 \text{ MB}$$

Stored as compressed WebP image crops averaging 45 KB each, the total exemplar disk image footprint is approximately 4.5 GB. This architectural decision allows the entire retrieval index to reside in high-speed host RAM, completely eliminating disk I/O bottlenecks during retrieval.

5.2 Visual Entity Linking: Parent-Child Chunking

In text-based RAG architectures, feeding an entire multi-page document into an embedding model dilutes key semantic signals. Similarly, passing an uncropped field photograph into a Vision Foundation Model introduces severe environmental noise. Background clutter – such as soil, sky, vegetation, water reflections, or human hands holding a specimen – dominates the receptive field, creating spurious cosine similarities between taxonomically distant species that inhabit similar macro-environments.

In order to isolate diagnostic visual signals, the system implements Parent-Child Visual Chunking:

- **Child Visual Chunk (I_{child}):** An upstream zero-shot detector (YOLO-World or OWL-ViT v2) scans the raw photograph to isolate the target specimen, outputting a normalised bounding box coordinates vector $B = [x_{\min}, y_{\min}, x_{\max}, y_{\max}]$. The organism is cropped, scaled to 224×224 pixels, and forwarded to the frozen feature extractor (BioCLIP-2) to produce the dense visual query embedding $z_{\text{child}} \in \mathbb{R}^{512}$:

$$z_{\text{child}} = f_{\text{VFM}}(I_{\text{child}}), \quad \|z_{\text{child}}\|_2 = 1$$

Where f_{VFM} represents the forward pass of the frozen vision backbone, and z_{child} is normalised to unit length on the hypersphere.

- **Parent Scene Context (I_{parent}):** The original uncropped field photograph is strictly preserved as an environmental habitat context layer. It is cryptographically linked to the child crop via a relational manifest tuple:

$$\text{EntityLink} = \langle \text{CropID}, \text{ParentURI}, B, \text{DarwinCorePayload} \rangle$$

During inference, vector similarity search operates strictly over the child visual embeddings (z_{child}). However, when exemplars are retrieved for downstream multimodal reasoning, both the child crop (fine morphology) and the parent scene (macro-habitat context) are injected into the context window. This elegantly prevents background clutter from distorting the nearest-neighbour search.

5.3 Vector Database: Single-Stage Filtered HNSW

Biological retrieval systems face a severe domain-specific failure mode known as *convergent evolution*. Under similar ecological pressures, unrelated organisms across distinct continents frequently evolve nearly identical visual morphologies. For example, the European Hornet (*Vespa crabro*) and the North American Cicada Killer (*Sphecius speciosus*) exhibit strikingly similar body geometry and colouration patterns.

If a retrieval engine implements standard two-stage post-filtering – performing a global nearest-neighbour vector search first, and then filtering results based on geographical metadata – the query will fail. If a user in Germany submits a photo of a European wasp, the top-50 visually

nearest neighbours returned by an unconstrained vector index might consist entirely of visually indistinguishable American species. Applying a subsequent geographical filter then drops all 50 candidates, resulting in an empty retrieval set and catastrophic context starvation.

In order to resolve this, the architecture adopts **Single-Stage Metadata-Filtered HNSW**. The Hierarchical Navigable Small World (HNSW) graph evaluates geographic and temporal boolean constraints natively *during* graph traversal, rather than after retrieval.

Let $\mathbf{x}_{\text{query}} = (\text{lat}_{\text{query}}, \text{lon}_{\text{query}})$ denote the GPS coordinates of the query, and $\tau_{\text{month}} \in \{1, \dots, 12\}$ denote the observation month. For each candidate node j encountered during the HNSW beam search, the traversal engine evaluates the deterministic predicate $\mathcal{P}(j)$:

$$\mathcal{P}(j) = [\mathcal{D}_{\text{geodesic}}(\mathbf{x}_{\text{query}}, \mathbf{x}_j) \leq R_{\text{max}}] \wedge [|\tau_{\text{month}} - \tau_j| \pmod{12} \leq \Delta_{\text{season}}]$$

Where:

- $\mathcal{D}_{\text{geodesic}}$ is the great-circle geographic distance between coordinate vectors, computed via the Haversine equation.
- $\mathbf{x}_j$ represents the historical observation coordinates associated with exemplar node j .
- R_{max} is the maximum admissible geographic search radius (for example, 500 km).
- τ_j is the recorded observation month of exemplar node j .
- Δ_{season} is the seasonal tolerance window ($\Delta_{\text{season}} = 1$, corresponding to a 3-month window centred on the query date).

If node j violates $\mathcal{P}(j)$, its outgoing edges are mathematically pruned from the current search path. This prevents the algorithm from descending into biogeographically invalid regions of the graph. This formulation ensures that the retrieved candidates are guaranteed to be both visually similar and physically plausible, thereby maintaining 100% valid retrieval recall in under 5 ms.

5.4 Conformal-Driven Lazy Execution

Executing zero-shot entity linking, vector graph traversal, and generative multimodal reasoning for every incoming request imposes prohibitive latency and computational costs. In order to maintain an SLA of under 25 ms for high-volume inference, the Vision RAG pipeline utilises conformal-driven lazy execution.

The incoming observation is first evaluated by the primary, lightweight INT8 ONNX classification head. The Split Conformal Prediction engine formally calculates the dynamic prediction set $\hat{C}(X)$ bounded by significance parameter $\alpha = 0.05$:

- **Decisive Prediction** ($|\hat{C}(X)| = 1$): The model exhibits high confidence in a single taxon. The identification is returned immediately to the client with sub-25ms latency. The Vision RAG pipeline remains completely idle, thus satisfying approximately 85% of production traffic at negligible compute cost.
- **Severe Ambiguity** ($|\hat{C}(X)| > k_{\text{max}}$): The model cannot distinguish between multiple closely related sibling species within the 95% confidence bound ($k_{\text{max}} = 3$). The system triggers the Vision RAG engine to retrieve parent-child exemplars for all taxa contained within $\hat{C}(X)$, explicitly providing context for comparative multimodal diagnosis.
- **Novelty and Out-of-Distribution** ($|\hat{C}(X)| = 0$): No known class meets the non-conformity threshold, signalling an anomalous, severely degraded, or uncatalogued taxon. The Vision RAG engine executes in unconstrained mode (with spatial filters relaxed), retrieving structural visual analogues across the global tree of life to assist taxonomic triage.

5.5 Multi-Modal Re-Ranking via Reciprocal Rank Fusion

Nearest-neighbour vector search establishes raw visual affinity, but visual embeddings alone cannot capture fine morphological traits, such as minor wing venation patterns or subtle genital plate structures. In order to enhance diagnostic relevance, retrieved candidates are re-ranked using Reciprocal Rank Fusion (RRF), integrating visual similarity with empirical biogeographical observation frequencies:

$$\text{Score}_{\text{RRF}}(i) = \frac{w_{\text{vis}}}{k + r_{\text{vis}}(i)} + \frac{w_{\text{geo}}}{k + r_{\text{geo}}(i)}$$

Where:

- $\text{Score}_{\text{RRF}}(i)$ is the consolidated ranking metric for candidate exemplar i .
- w_{vis} is the scalar weighting factor assigned to visual cosine similarity ($w_{\text{vis}} = 0.65$).
- w_{geo} is the scalar weighting factor assigned to the biogeographical occurrence prior ($w_{\text{geo}} = 0.35$).
- k is a rank-smoothing constant ($k = 60$) that prevents low-ranking outliers from dominating the fused score.
- $r_{\text{vis}}(i)$ is the ordinal rank ($1, 2, 3, \dots$) of candidate i sorted descending by cosine similarity $z_{\text{child}} \cdot z_i$.
- $r_{\text{geo}}(i)$ is the ordinal rank of candidate i sorted descending by historical observation density $P(y_i \mid \text{Cell}_{\text{H3}}, \tau_{\text{month}})$, derived from pre-indexed GBIF occurrence frequencies.

This rank fusion guarantees that candidates appearing at the top of the context window are both visually representative of the query and frequently documented within that ecological spatial-temporal niche.

5.6 Multimodal Context Synthesis & VLM Diagnostics

The top-3 candidates resulting from Reciprocal Rank Fusion are assembled into a structured prompt schema along with the query observation. This payload is dispatched to a localised Vision-Language Model (such as a quantised Qwen2-VL-2B instance running on host P-cores or the RTX 5070 Ti).

In order to ensure machine readability and eliminate conversational hallucination, the VLM is constrained by grammar-based sampling to emit a strictly typed JSON payload:

```

1 {
2   "conformal_context_size": 2,
3   "candidates_evaluated": ["Bombus terrestris", "Bombus lucorum"],
4   "primary_discriminator": "Pronotal collar coloration and width",
5   "morphological_analysis": "The specimen exhibits a dark buff-yellow abdominal band and
6   ↪ brownish-white tail hairs matching Bombus terrestris rather than the bright lemon-yellow band
7   ↪ typical of Bombus lucorum.",
8   "recommended_taxon": "Bombus terrestris",
9   "epistemic_verdict": "HUMAN_TRIAGE_RECOMMENDED",
10  "rationale": "High visual overlap under strong sunlight reflection; wing marginal cell venation
11  ↪ partially occluded."
12 }
```

This structured response is rendered on the HTMX operator dashboard as a comparative differential diagnosis card. This allows human specialists to inspect aligned visual crops alongside identified discriminative keys.

5.7 Closed-Loop Active Learning & Taxonomic Mutations

The Vision RAG infrastructure acts as the primary data ingestion engine for human-in-the-loop active learning workflows. When the VLM returns an epistemic verdict of `HUMAN_TRIAGE_RECOMMENDED`, the observation and its candidate context are mirrored directly to the DagsHub Label Studio workspace. Expert annotations subsequently flow back into the system, expanding the training manifests and updating the reference exemplar index.

Furthermore, biological classification systems frequently undergo Linnaean realignments:

- **Taxonomic Split:** When a recognised species is divided into two distinct taxa, historical vectors in the HNSW database matching the parent taxon are flagged with the metadata attribute `requires_reannotation`. The system gracefully routes these exemplars to Label Studio for expert sorting, purging obsolete nodes from the active index without requiring neural network retraining.
- **Taxonomic Lump:** When two distinct taxa are combined into a single unified species, the system performs an in-place metadata alias update on the vector node payloads. The underlying embedding vectors remain mathematically valid, thereby updating the taxonomic knowledge base in milliseconds.

5.8 End-to-End Latency & Resource Allocation

The operational performance of the dual-path inference engine is bound by strict execution latency and memory constraints, as detailed in Table 5.1.

Table 5.1: End-to-End Latency and Resource Budget SLA

Pipeline Segment	Execution Engine	Latency SLA	Memory Footprint
Zero-Shot Cropping	YOLO-World (ORT INT8)	12.4 ms	380 MB VRAM
VFM Feature Extractor	BioCLIP-2 (ORT FP16)	8.2 ms	1,120 MB VRAM
Conformal Gate	PyTorch / C-Array	0.8 ms	< 50 MB RAM
Filtered HNSW	Embedded HNSWlib	3.5 ms	205 MB RAM
Multi-Modal RRF	In-Memory Polars / DuckDB	1.2 ms	< 20 MB RAM
VLM Synthesis (Lazy)	Quantised 2B VLM	280.0 ms	2,200 MB VRAM
Fast Hot-Path (85%)	Direct ONNX Inference	21.4 ms	1,550 MB VRAM
Lazy RAG Path (15%)	Full Retrieval + VLM	306.1 ms	3,750 MB VRAM

Chapter 6

Epistemic Safety & Active Learning

Deploying deep neural networks in automated ecological workflows introduces a fundamental operational risk: conventional classifiers are notoriously uncalibrated and prone to overconfident hallucinations. Under standard training objectives, a convolutional network or vision transformer produces normalised output vectors via the softmax operator. When presented with blurred imagery, non-biological artefacts, or previously uncatalogued taxa, these models frequently assign high confidence to incorrect categories. In biodiversity monitoring, silent misclassifications corrupt environmental databases and ultimately undermine conservation interventions.

In order to establish genuine production reliability, our serving architecture mathematically decouples feature extraction from raw point predictions. The platform combines energy-based density filtering, deterministic spatial-temporal constraint masking, and distribution-free Split Conformal Prediction into an integrated epistemic safety layer. This pipeline guarantees bounded error rates under arbitrary data distributions and programmatically routes ambiguous observations into human-in-the-loop active learning workflows.

6.1 Energy-Based Out-of-Distribution Detection

Field deployments continuously encounter non-biological clutter, corrupt sensor transmissions, and organisms belonging to unmodelled phyla. Evaluating uncertainty sets on such inputs is computationally wasteful and pollutes downstream monitoring metrics. Consequently, the first stage of the safety pipeline intercepts and filters out-of-distribution (OOD) observations directly in the feature space prior to probability normalisation.

Traditional methods rely on the Maximum Softmax Probability (MSP), $\max_k \hat{\pi}_k(x)$, which suffers from high false-positive rates due to the normalisation constraint forcing probabilities to sum to one regardless of absolute logit magnitudes. Instead, the platform derives an unconstrained free energy score directly from the logit outputs. Conceptually, Helmholtz free energy measures the physical compatibility of an input with the learned statistical parameters of the model. In-distribution samples occupy high-density regions characterised by lower energy states, whereas anomalous samples inhabit low-density regions characterised by elevated free energy.

Formally, consider an input image vector $x \in \mathcal{X}$ processed by a backbone feature extractor producing an unnormalised logit vector $f(x) \in \mathbb{R}^K$, where K denotes the total number of modelled taxa. For any logit coordinate $f_j(x)$ corresponding to class index $j \in \{1, \dots, K\}$, the Gibbs-Boltzmann distribution links the class probability to the scalar energy function $E(x; f)$ via temperature scaling

parameter $T \in \mathbb{R}^+$:

$$E(x; f) = -T \cdot \log \sum_{j=1}^K \exp \left(\frac{f_j(x)}{T} \right)$$

In this formulation:

- x represents the normalised child visual crop passed to the network.
- f denotes the parameterised mapping from pixel space to the logit representation.
- $f_j(x)$ is the scalar logit score assigned by the model to taxon j .
- K is the discrete cardinality of the operational taxonomy.
- T is the temperature scaling hyperparameter ($T > 0$) that controls the smoothness of the log-sum-exp surface.
- $E(x; f)$ is the resulting scalar energy score.

During inference, the system evaluates $E(x; f)$ against an empirically calibrated decision threshold $\tau_{\text{OOD}} \in \mathbb{R}$. The decision rule $\Phi(x)$ gates subsequent pipeline execution:

$$\Phi(x) = \begin{cases} 1 & \text{(In-Distribution: Proceed to Spatial Masking), if } E(x; f) \leq \tau_{\text{OOD}} \\ 0 & \text{(Out-of-Distribution: Reject Immediately), if } E(x; f) > \tau_{\text{OOD}} \end{cases}$$

The threshold τ_{OOD} is strictly tuned on an empirical holdout split to capture 95% of in-distribution validation observations whilst maximising Precision-Recall AUC (PR-AUC) against negative control datasets containing empty foliage and mechanical objects. Observations failing this check bypass classification entirely, thus preventing ungrounded inference.

6.2 Spatial-Temporal Constraint Masking

Visual similarity search in high-dimensional embedding spaces frequently encounters convergent evolution, where distinct organisms inhabiting disparate continents develop overlapping morphological features. While a purely visual classifier may assign comparable likelihoods to such species, physical geography imposes strict physical boundaries.

In order to enforce ecological validity without incurring the computational overhead of additional deep learning models, the serving engine applies deterministic boolean masks directly to the post-OOD logit vector. Every citizen-science observation arrives with DarwinCore spatial-temporal metadata: latitude, longitude, and calendar timestamp. The ingestion pipeline systematically maps the observation coordinates to an Uber H3 hexagonal spatial cell index $h \in \mathcal{H}$ and maps the timestamp to an observation month $m \in \{1, \dots, 12\}$.

Let $\mathcal{A}(h, m) \subseteq \{1, \dots, K\}$ define the admissible species set, constructed from verified historical GBIF occurrences within a geodesic radius R of spatial cell h during seasonal window $m \pm 1$. The masked logit vector $\tilde{f}(x) \in \mathbb{R}^K$ is defined element-wise for each taxon $j \in \{1, \dots, K\}$ as follows:

$$\tilde{f}_j(x) = \begin{cases} f_j(x), & \text{if } j \in \mathcal{A}(h, m) \\ -\infty, & \text{if } j \notin \mathcal{A}(h, m) \end{cases}$$

Applying the softmax function over $\tilde{f}(x)$ yields the biogeographically constrained probability distribution $\hat{\pi}(x) \in [0, 1]^K$:

$$\hat{\pi}_j(x) = \frac{\exp(\tilde{f}_j(x))}{\sum_{k=1}^K \exp(\tilde{f}_k(x))}$$

Where $\hat{\pi}_j(x)$ represents the estimated probability that input x belongs to taxon j . By forcing $\tilde{f}_j(x) = -\infty$ for biogeographically impossible taxa, $\hat{\pi}_j(x)$ evaluates strictly to zero. This ensures that geographically impossible candidate species are pruned before uncertainty sets are ever constructed.

6.3 Distribution-Free Uncertainty via Split Conformal Prediction

Point predictions derived from $\arg \max_j \hat{\pi}_j(x)$ provide no statistical reliability guarantees. In mission-critical biodiversity intelligence, a system must convey not only its best guess, but also the statistical confidence bounded by an explicit error rate. In order to achieve this, the platform implements Split Conformal Prediction. This converts raw probability distributions into dynamic prediction sets guaranteed to contain the ground-truth taxon under exchangeability assumptions.

6.3.1 Calibration Protocol and Quantile Estimation

Let $\mathcal{D}_{\text{cal}} = \{(x_i, y_i)\}_{i=1}^n$ represent an independent calibration dataset containing n exchangeable observations, completely disjoint from the model training and validation splits. For each calibration observation (x_i, y_i) , where x_i is the visual crop and $y_i \in \{1, \dots, K\}$ is the ground-truth Linnaean class label, the pipeline computes a scalar non-conformity score $s_i \in [0, 1]$:

$$s_i = 1 - \hat{\pi}_{y_i}(x_i)$$

In this expression:

- x_i is the i -th calibration image.
- y_i is the confirmed ground-truth class label for observation i .
- $\hat{\pi}_{y_i}(x_i)$ is the model's constrained softmax probability assigned to the correct class y_i .
- s_i denotes the non-conformity score, which increases towards 1 as model confidence in the true label degrades.

Given a user-specified significance level $\alpha \in (0, 1)$ corresponding to a targeted statistical coverage level of $1 - \alpha$ (for example, $\alpha = 0.05$ for 95% coverage), the system computes the empirical quantile threshold $\hat{q}_\alpha$ over the sorted non-conformity scores $s_{(1)} \leq s_{(2)} \leq \dots \leq s_{(n)}$:

$$\hat{q}_\alpha = \text{Quantile} \left(\{s_1, \dots, s_n\}; \frac{\lceil (n+1)(1-\alpha) \rceil}{n} \right) = s_{(\lceil (n+1)(1-\alpha) \rceil)}$$

Here, $\lceil \cdot \rceil$ represents the ceiling function, thereby ensuring that the empirical quantile conservatively guards the discrete calibration sample distribution against finite-sample bias.

6.3.2 Prediction Set Construction & Marginal Coverage Guarantee

During real-time inference on an unseen query photograph X_{n+1} with unknown true label Y_{n+1} , the system evaluates candidate taxa across the taxonomy. The prediction set $\hat{C}(X_{n+1}) \subseteq \{1, \dots, K\}$ is constructed by including all taxa whose non-conformity scores fall below the calibrated threshold $\hat{q}_\alpha$:

$$\hat{C}(X_{n+1}) = \{y \in \{1, \dots, K\} : 1 - \hat{\pi}_y(X_{n+1}) \leq \hat{q}_\alpha\} = \{y \in \{1, \dots, K\} : \hat{\pi}_y(X_{n+1}) \geq 1 - \hat{q}_\alpha\}$$

In this set definition:

- X_{n+1} represents the unseen test observation input.
- Y_{n+1} denotes the unobserved ground-truth species identifier for X_{n+1} .
- $\hat{C}(X_{n+1})$ is the output prediction set containing zero, one, or multiple candidate taxa.
- $\hat{\pi}_y(X_{n+1})$ is the model's biogeographically filtered probability estimate for candidate species y .
- $\hat{q}_\alpha$ is the scalar calibration threshold computed from $\mathcal{D}_{\text{cal}}$.

Because the calibration observations and test observations are mathematically exchangeable, the conformal coverage theorem guarantees finite-sample marginal validity:

$$P\left(Y_{n+1} \in \hat{C}(X_{n+1})\right) \geq 1 - \alpha$$

Crucially, this guarantee is completely distribution-free. It holds without requiring parametric assumptions about the underlying image distributions, visual features, or taxonomic frequencies. The prediction set cardinality dynamically scales with image ambiguity: clean, prototypical observations yield tight, single-element sets, whereas degraded or visually ambiguous observations yield expanded sets that reflect inherent epistemic uncertainty.

6.4 Conformal Triage Routing & Active Learning Feedback

The cardinality of the conformal prediction set, $|\hat{C}(X_{n+1})|$, provides an automated, mathematically grounded signal for operational triage. The system maps the set size directly to service routing logic:

1. **Decisive Prediction** ($|\hat{C}| = 1$): The prediction set contains exactly one species candidate that satisfies the $1 - \alpha$ confidence threshold. The identification is deemed statistically conclusive and is returned to the user via the FastAPI prediction endpoint with sub-25ms latency. No human review or secondary model evaluation is triggered.
2. **Severe Classification Ambiguity** ($|\hat{C}| > k_{\max}$): The model cannot distinguish between multiple sibling taxa within the specified error bound, indicating high visual overlap or extreme environmental noise. When $|\hat{C}|$ exceeds a predefined threshold $k_{\max}$ (typically $k_{\max} = 3$), automated publication halts, and the query is diverted to the expert verification queue.
3. **Novelty and Out-of-Distribution Remnants** ($|\hat{C}| = 0$): No candidate species achieved a non-conformity score below $\hat{q}_\alpha$. This occurs when an input organism diverges significantly from all known calibration distributions, signalling anomalous specimens, unmodelled long-tail taxa, or novel regional invasive species. These empty sets bypass standard workflows and are flagged directly for botanical or zoological taxonomy review.

6.4.1 Active Learning Query Strategies

Observations flagged with $|\hat{C}| > k_{\max}$ or $|\hat{C}| = 0$ are synchronised to a managed DagsHub Label Studio instance for human-in-the-loop review. However, because professional taxonomic annotation is an expensive operational resource, the triage queue prioritises pending samples using active learning algorithms rather than simple first-in, first-out ordering.

The pipeline implements Batch Active Learning by Diverse Gradient Embeddings (BADGE). For each unannotated query x , BADGE models epistemic uncertainty by calculating the gradient of the cross-entropy loss with respect to the parameters of the final linear classification layer, θ_{last} , evaluated against the model’s current predicted pseudo-label $\hat{y} = \arg \max_j \hat{\pi}_j(x)$:

$$g_x = \nabla_{\theta_{\text{last}}} \mathcal{L}(f(x; \theta), \hat{y})$$

Where:

- $g_x \in \mathbb{R}^{d \times K}$ is the resulting gradient embedding vector for candidate image x .
- d is the dimensionality of the penultimate feature embedding vector ($d = 512$ for BioCLIP-2).
- $\mathcal{L}$ represents the empirical loss objective.
- $\hat{y}$ is the model’s top-1 pseudo-label.

The magnitude of g_x reflects model uncertainty, whilst its directional vector in $\mathbb{R}^{d \times K}$ reflects the nature of parameter adjustments required. The active learning engine applies k -means++

seeding over the pool of gradient vectors $\{g_x\}$ in order to select a diverse batch of high-uncertainty observations.

Validated annotations from Label Studio are ingested into the data pipeline on a scheduled cadence. Newly confirmed samples are added to the DVC-versioned dataset registry, expanding the training split for rare long-tail species and updating the holdout calibration split $\mathcal{D}_{\text{cal}}$. This directly drives continuous training workflows, progressively compressing average conformal set sizes across successive model versions whilst maintaining strict coverage invariants.

Chapter 7

Observability, Testing & Deployment

Transitioning a deep learning system from a localised development environment into a production ecosystem fundamentally shifts the engineering challenge from model optimisation to system verification, runtime isolation, and continuous operational health monitoring. A machine learning service rarely fails through overt code exceptions; instead, it fails silently through data distribution shift, memory leaks under continuous streaming, Translation Lookaside Buffer thrashing, and uncalibrated epistemic decay.

In order to maintain production integrity, our deployment architecture rigorously couples containerised runtime encapsulation with Kubernetes cluster orchestration, real-time statistical telemetry, and a multi-tiered verification framework. This chapter outlines the software engineering mechanisms governing containerisation, traffic routing, automated drift alerting, and offline testing suites.

7.1 Containerisation and Multi-Stage Runtimes

Standard Python environments managed via virtual environment tools are notoriously fragile in production. Differences in dynamically linked system libraries, CUDA driver mismatches on host kernels, and unversioned C-extension compilation dependencies frequently cause models to behave non-deterministically across infrastructure boundaries.

In order to guarantee bit-exact execution determinism, the platform carefully packages the inference engine into an immutable container image using a multi-stage Docker build pipeline. This process cleanly isolates the build environment from the final execution runtime, thereby drastically shrinking the attack surface and image size.

7.1.1 Multi-Stage Image Compilation

The build pipeline is logically partitioned into two distinct stages:

1. **The Builder Stage:** Utilises a full Linux distribution equipped with C++ compilation toolchains, NVIDIA CUDA development toolkits, and the Pixi package manager. In this stage, Pixi elegantly resolves the exact dependency graph specified in `pyproject.toml` and locked in `pixi.lock`, compiling native C-extensions and pre-building optimised ONNX Runtime binaries.
2. **The Runtime Stage:** A minimal, distroless Debian-slim base image is initialised, containing strictly the shared Linux kernel interfaces and runtime CUDA execution libraries. The compiled virtual environment, static assets, and FastAPI service code are subsequently copied

from the builder stage without carrying over compilers, debugging symbols, or package caches.

7.1.2 Kernel-Level Process Isolation

At runtime, execution security and resource determinism are enforced directly through robust Linux kernel primitives:

- **Namespaces:** The containerised process is systematically isolated across six discrete namespaces: Process ID (PID) isolation prevents inspection of host workloads, Network (NET) virtualisation seals internal socket bindings, Mount (MNT) controls filesystem scoping, and Inter-Process Communication (IPC) strictly prevents unauthorised shared memory access.
- **Control Groups (cgroups v2):** Hardware utilisation boundaries are enforced at the kernel boundary. The container runtime configures hard memory ceilings to entirely eliminate out-of-memory cascading failures and carefully pins CPU thread execution to prevent thread migration away from dedicated host performance cores.
- **Rootless Execution:** The container runs strictly under an unprivileged user namespace (USER `appuser`), mounting the root filesystem as read-only and intelligently routing temporary scratch buffers to an ephemeral, in-memory `tmpfs` volume.

7.2 Kubernetes Cluster Orchestration and Workload Routing

Deploying a hybrid vision architecture – where routine queries resolve in sub-25ms hot paths whilst ambiguous queries trigger heavy generative multimodal retrieval – requires cluster orchestration capable of handling highly asymmetric latency profiles. The system is deployed across a managed Kubernetes cluster strictly configured to separate lightweight inference paths from resource-intensive Vision RAG pipelines.

7.2.1 Asymmetric Pod Architecture and Ingress Routing

Rather than packaging the fast-path ONNX classifier and the multi-gigabyte Vision-Language Model into a single monolithic pod, the cluster architecture deliberately splits the workload into two specialised microservices:

1. **The Primary Classifier Service (`taxon-vision-serving`):** A high-density, horizontally scaled deployment of lightweight FastAPI pods hosting the INT8 quantised ONNX backbone and the Split Conformal Prediction engine. These pods require minimal VRAM (≈ 1.5 GB) and handle 100% of initial incoming traffic, rapidly resolving 85% of queries in under 22 ms.
2. **The Multimodal RAG Service (`taxon-vision-vlm`):** A specialised, resource-allocated deployment mounting dedicated GPU Tensor Cores to host the quantised 2-billion parameter Vision-Language Model and the in-memory HNSW vector index.

An ingress controller inspects incoming network payloads and intelligently load-balances observation traffic across the `taxon-vision-serving` pods. If an observation’s conformal prediction set cardinality evaluates to $|C| = 1$, the primary pod returns the classification directly through the ingress to the client. If the conformal layer flags ambiguity ($|C| > 1$) or novelty ($|C| = 0$), the primary service seamlessly dispatches an asynchronous internal gRPC call to the `taxon-vision-vlm` service. This service then executes filtered graph traversal, synthesises the comparative diagnostic context, and efficiently returns the structured differential payload.

7.2.2 Self-Healing and Health Probing

In order to prevent cluster degradation caused by deadlocks or memory corruption, Kubernetes enforces dual-tier health probes:

- **Liveness Probes:** The kubelet periodically polls a lightweight `/healthz/liveness` HTTP endpoint. If the FastAPI event loop becomes unresponsive or encounters an unrecoverable worker thread failure, the control plane immediately terminates the failing container and instantiates a clean replica.
- **Readiness Probes:** A separate `/healthz/readiness` probe strictly verifies that internal model buffers, CUDA execution providers, and HNSW graph indexes are fully loaded into memory before the ingress controller ever routes live citizen-science observation traffic to the pod.

7.2.3 Horizontal Pod Autoscaling on Custom Telemetry

Standard Kubernetes autoscaling typically relies on CPU or system memory thresholds, which unfortunately provide misleading operational signals for neural network workloads. A GPU-bound container can experience severe execution queuing whilst reporting surprisingly low CPU utilisation.

The cluster implements Horizontal Pod Autoscaling (HPA) driven by custom Prometheus metrics via the Kubernetes Custom Metrics API. The autoscaler continuously monitors two primary indicators:

1. **GPU VRAM Saturation and Compute Utilisation:** Sourced directly from NVIDIA Data Center GPU Manager (DCGM) metrics.
2. **Conformal Ambiguity Influx:** Sourced directly from the application’s telemetry. If an environmental event (for example, sudden regional weather shifts causing blurred imagery) causes the proportion of multi-candidate conformal sets ($|C| > 1$) to spike, the HPA preemptively scales the `taxon-vision-vlm` pod pool to comfortably absorb the anticipated surge in heavy multimodal retrieval passes before service latency inevitably degrades.

7.3 Telemetry, Metric Exporters, and Concept Drift Monitoring

Unlike traditional software services that experience deterministic failure modes, machine learning systems are uniquely vulnerable to silent statistical degradation. A model endpoint can report 100% uptime and sub-millisecond HTTP response times whilst concurrently outputting meaningless ecological predictions due to seasonal distribution shifts, camera sensor variations, or geographic migrations. Thus, the platform embeds comprehensive statistical telemetry deeply into the serving runtime.

7.3.1 Prometheus Telemetry Instrumentation

The FastAPI application exposes a secure `/metrics` endpoint scraped at regular intervals by a Prometheus time-series collector. Monitored indicators comprehensively span system infrastructure, computational latency, and model decision distributions:

- **Inference Latency Histograms (`inference_latency_seconds_bucket`):** High-resolution histograms tracking the duration of zero-shot cropping, ONNX backbone execution, and conformal set evaluation, effectively generating empirical p50, p95, and p99 latency distributions.
- **Conformal Prediction Set Distribution (`conformal_set_size`):** A sliding-window metric tracking the cardinality distribution of $\hat{C}(X)$. Under stationary distribution conditions, the average set size $\mathbb{E}[|\hat{C}|]$ remains fundamentally constant. A measured expansion in set size mathematically indicates rising classification ambiguity.
- **Energy OOD Rejection Counter (`ood_rejections_total`):** Tracks the cumulative volume and rate of incoming images rejected by the free-energy scoring layer ($E(x; f) > \tau_{\text{OOD}}$).

- **Tail-Species Classification Gauge (`tail_species_accuracy`):** Monitors real-time identification accuracy on verified long-tail species subsets possessing fewer than 20 historical training observations.

7.3.2 Mathematical Formulation of Feature Drift via Maximum Mean Discrepancy

In order to detect subtle visual distribution shifts before they manifest as catastrophic classification errors, the monitoring pipeline mathematically evaluates high-dimensional feature drift over the penultimate embedding space $\mathbb{R}^d$.

Let $\mathcal{Z}_{\text{base}} = \{z_1, \dots, z_M\}$ denote a reference baseline distribution of M feature vectors gracefully extracted from the holdout validation split during model calibration. Let $\mathcal{Z}_{\text{prod}} = \{z'_1, \dots, z'_N\}$ denote a streaming batch of N feature vectors collected from live production inference over a rolling temporal window W .

The system formally computes the Maximum Mean Discrepancy (MMD), a non-parametric metric evaluating the distance between two probability distributions within a Reproducing Kernel Hilbert Space (RKHS). Formally, the empirical squared MMD is calculated as follows:

$$\text{MMD}^2(\mathcal{Z}_{\text{base}}, \mathcal{Z}_{\text{prod}}) = \frac{1}{M^2} \sum_{i=1}^M \sum_{j=1}^M k(z_i, z_j) - \frac{2}{MN} \sum_{i=1}^M \sum_{j=1}^N k(z_i, z'_j) + \frac{1}{N^2} \sum_{i=1}^N \sum_{j=1}^N k(z'_i, z'_j)$$

Where:

- $\mathcal{Z}_{\text{base}}$ is the set of M baseline visual embeddings cleanly cached during offline calibration.
- $\mathcal{Z}_{\text{prod}}$ is the set of N production visual embeddings observed during live operation.
- M and N denote the sample cardinalities of the baseline and production splits, respectively.
- $z_i, z_j \in \mathcal{Z}_{\text{base}}$ represent individual reference embedding vectors.
- $z'_i, z'_j \in \mathcal{Z}_{\text{prod}}$ represent individual live operational embedding vectors.
- $k(z, z')$ is a Gaussian Radial Basis Function (RBF) kernel defined as:

$$k(z, z') = \exp\left(-\frac{\|z - z'\|_2^2}{2\sigma^2}\right)$$

With bandwidth hyperparameter $\sigma \in \mathbb{R}^+$ controlling the sensitivity of the spatial mapping in feature space.

7.3.3 Automated Alerting and Continuous Retraining Triggers

Prometheus Alertmanager evaluates streaming metrics directly against hard operational invariants. Two primary conditions trigger automated intervention:

1. **Conformal Inflation Alert:** Triggered if the mean prediction set cardinality $\mathbb{E}[|\hat{C}|]$ expands by more than 20% over a rolling 1-hour window relative to baseline calibration distributions.
2. **Embedding Drift Alert:** Triggered if the feature space distribution distance securely exceeds an empirical threshold, $\text{MMD}^2(\mathcal{Z}_{\text{base}}, \mathcal{Z}_{\text{prod}}) > \tau_{\text{drift}}$.

When an alert inevitably fires, Alertmanager dispatches an authenticated webhook payload to the GitHub Actions repository gateway (`repository_dispatch` event). This initiates an automated Level 3 continuous training workflow: the local bare-metal runner pulls the newest validated DVC dataset snapshot, retrains the classification head with Class-Balanced Loss, rigidly re-calibrates the conformal quantile $\hat{q}_\alpha$, executes regression invariant tests, and finally opens a staging Pull Request containing the comparative telemetry plots for engineering sign-off.

7.4 Multi-Tier Verification and Testing Strategy

A continuous integration pipeline for machine learning must validate considerably more than simple syntax correctness. The TaxonVision-MLOps testing framework strictly enforces a four-tiered verification suite executing on every commit.

7.4.1 Tier 1: Upstream API Mocking

During automated CI builds, executing real network requests against the live iNaturalist API violates rate-limiting policies and inevitably introduces non-deterministic network latency.

The test harness uses the **responses** library to gracefully intercept outgoing HTTP calls at the transport layer. The test runner injects mock JSON responses simulating diverse API states: valid DarwinCore metadata payloads, rate-limited responses (HTTP 429), server timeout conditions (HTTP 504), and malformed taxonomic trees. This rigorously guarantees that ingestion, licence filtering, and coordinate parsing logic are safely validated in under 5 seconds without any network dependencies.

7.4.2 Tier 2: Object Storage and Pipeline Mocking

In order to test Data Version Control (DVC) pipelines and S3 streaming operations without incurring AWS billing or requiring live DagsHub credentials, the integration suite happily employs the **moto** testing library.

The test setup intelligently instantiates an in-memory, ephemeral AWS S3 service directly within the local pytest process. The DVC client is pointed towards this virtual endpoint, readily allowing the test runner to strictly verify cryptographic hash tracking, multi-part binary streaming, and dataset version switching entirely in local memory.

7.4.3 Tier 3: Mathematical Invariant Testing

Standard software testing relies heavily on binary assertions (for example, verifying an output equals a known constant). Machine learning models, however, are inherently probabilistic. The pipeline introduces robust mathematical invariant tests that evaluate statistical coverage guarantees well before model serialisation.

The test runner dutifully loads the newly trained ONNX candidate model and evaluates it against an unseen holdout test split $\mathcal{D}_{\text{test}} = \{(x_i, y_i)\}_{i=1}^{n_{\text{test}}}$. The test formally asserts that the empirical coverage mathematically satisfies the statistical bound:

$$\text{Empirical Coverage} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} \mathbb{I}(y_i \in \hat{C}(x_i)) \geq 1 - \alpha$$

Where:

- n_{test} is the total number of evaluation observations in the holdout split.
- (x_i, y_i) denotes the i -th test image and its corresponding ground-truth Linnaean label.
- $\hat{C}(x_i)$ is the prediction set output by the conformal engine evaluated at threshold $\hat{q}_\alpha$.
- $\mathbb{I}(\cdot)$ is the indicator function, evaluating to 1 if the true label y_i is safely contained within $\hat{C}(x_i)$, and 0 otherwise.
- α is the nominal significance error rate ($\alpha = 0.05$).

If the empirical coverage drops below 0.95, the test fails with a fatal error. This mathematically prevents an under-covered, uncalibrated model from ever being packaged into a release artefact.

7.4.4 Tier 4: Statistical Metric Selection and Anomaly Verification

In traditional computer vision benchmarking, the Area Under the Receiver Operating Characteristic curve (AUROC) is widely utilised to quantify model discrimination. In biological species monitoring and ecological anomaly detection, however, **AUROC is fundamentally flawed and actively deceptive**.

The Failure of AUROC under Ecological Class Imbalance

Biological datasets are heavily governed by extreme power-law distributions: common species possess tens of thousands of observations, whilst rare long-tail species and genuine out-of-distribution anomalies account for a minuscule fraction of the data.

The AUROC metric mathematically evaluates the trade-off between the True Positive Rate (TPR) and the False Positive Rate (FPR):

$$TPR = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN}$$

Where TP , FP , TN , and FN denote the counts of true positives, false positives, true negatives, and false negatives, respectively.

Because the negative class (normal, common organisms or background pixels) vastly outnumbers the anomalous class, the denominator of the False Positive Rate ($FP + TN$) is aggressively dominated by TN . Consequently, an algorithm can easily generate thousands of false positive alarms (FP), yet the calculated FPR remains near zero due to the massive TN divisor. A model exhibiting an apparently stellar AUROC score of 0.98 can simultaneously suffer a catastrophic precision collapse in production, inundating human experts with hundreds of useless false alerts.

Mandating PR-AUC and AUPIMO

In order to permanently prevent misleading verification gates, the testing suite explicitly rejects AUROC in favour of Precision-Recall Area Under the Curve (PR-AUC) and Area Under the Per-Image Overlap (AUPIMO).

For classification and global Out-of-Distribution anomaly detection, the pipeline robustly evaluates PR-AUC:

$$\text{PR-AUC} = \int_0^1 \text{Precision}(R) dR$$

Where Precision is formally defined as $\frac{TP}{TP+FP}$ and Recall (R) is defined as $\frac{TP}{TP+FN}$. Because PR-AUC mathematically excludes True Negatives (TN) from its formulation entirely, it directly reflects the true proportion of genuine biological anomalies amongst all flagged alerts, thus providing an honest measure of real-world efficacy.

For zero-shot visual cropping, bounding-box isolation, and spatial anomaly segmentation, the test harness enforces the Area Under the Per-Image Overlap (AUPIMO) metric. Unlike global pixel-level metrics that carelessly allow large contiguous background areas to distort evaluation scores, AUPIMO computes the intersection-over-union metric independently across each individual image's bounding box over varying threshold levels:

$$\text{AUPIMO} = \frac{1}{N_{\text{images}}} \sum_{k=1}^{N_{\text{images}}} \int_0^{\tau_{\text{max}}} \Omega_k(\tau) d\tau$$

Where:

- N_{images} is the total number of test photographs carefully evaluated in the spatial verification suite.
- τ denotes the detection threshold, integrated from 0 to an upper operational bound τ_{max} .

- $\Omega_k(\tau)$ represents the per-image overlap rate for photograph k evaluated at threshold τ , defined precisely as the ratio of correctly identified anomaly pixels within the ground-truth organism mask relative to the total organism area.

By thoughtfully evaluating overlap per image rather than across the pooled pixel space, AUPIMO ensures that the zero-shot visual entity linking module isolates small organisms (for example, small insects or solitary flowers) just as reliably as large organisms (such as large mammals or mature trees). This mathematical guarantee ensures that upstream cropping logic introduces zero systematic taxonomic bias into downstream retrieval.

Chapter 8

Hardware Optimisation and Memory Mathematics

Operating a high-throughput machine learning service under strict latency service-level agreements (SLAs) requires moving decisively beyond theoretical model performance and addressing the physical limitations of silicon. This chapter explicitly defines the mathematical transformations utilised to compress neural network memory footprints, the formal theorems rigorously governing epistemic safety thresholds, and the memory complexity equations required to safely bound vector database consumption.

8.1 Dynamic Quantisation and Tensor Compression

Deep neural networks are fundamentally composed of dense, high-dimensional weight matrices. By default, standard training frameworks construct these matrices using 32-bit floating-point (FP32) numerical representations. Whilst FP32 provides extreme gradient precision during the backpropagation phase of training, it is highly inefficient during production inference. FP32 tensors consume vast amounts of GPU VRAM, and executing matrix multiplications at this precision severely bottlenecks the memory bandwidth, thereby increasing prediction latency.

In order to optimise the deployment architecture, the PyTorch classification head and feature extractor are exported to the Open Neural Network Exchange (ONNX) format, cleanly decoupling the computational graph from Python’s execution overhead. Subsequently, the ONNX Runtime strategically applies Integer-8 (INT8) dynamic quantisation. This mathematical operation robustly maps the high-resolution continuous floating-point weights onto a discrete 8-bit integer grid.

The quantisation mapping function is formally defined as:

$$q = \text{round}\left(\frac{x}{S}\right) + Z$$

In this formulation:

- q represents the resulting 8-bit quantised integer representation.
- x denotes the original 32-bit floating-point weight or activation value from the trained network.
- S is the floating-point scale factor, which effectively defines the continuous step size between discrete quantised bins.
- Z is the 8-bit integer zero-point offset, which aligns the real-world mathematical zero with the quantised integer zero in order to ensure zero-padding operations remain absolutely precise.

By mapping 32-bit floats to 8-bit integers, the model’s memory footprint is deterministically compressed by 75% (for example, reducing a 400 MB architecture to approximately 100 MB). Because INT8 operands are significantly smaller, modern hardware can efficiently pack multiple operations into single SIMD (Single Instruction, Multiple Data) registers, strongly leveraging AVX-512 Vector Neural Network Instructions (VNNI) on CPUs or Tensor Cores on GPUs. This delivers a $2\times$ to $4\times$ acceleration in inference latency whilst historically yielding less than a 1% degradation in top-1 accuracy.

8.2 Marginal Coverage Guarantees in Conformal Prediction

Standard classification models merely output uncalibrated probability distributions. In a biological monitoring context, an uncalibrated model might confidently assign a 99% softmax probability to a specific species of tiger, even when presented with a heavily obscured photograph of a domestic cat. Relying on these raw probability scores inevitably introduces unacceptable epistemic risk into the data pipeline.

In order to thoroughly mitigate this, the architecture securely wraps the base classifier in a Split Conformal Prediction layer. Rather than forcing the model to emit a single, highly confident – yet potentially entirely incorrect – point prediction, conformal prediction explicitly evaluates the input ambiguity and generates a dynamic *set* of candidate taxa. If the visual input is clean and prototypical, the set cleanly collapses to a single species. If the image is blurry, occluded, or visually ambiguous, the set naturally expands (for instance, returning three visually similar sibling species) to maintain absolute mathematical safety.

Formally, this epistemic safety is guaranteed by the marginal coverage theorem:

$$\mathbb{P}(Y_{n+1} \in \hat{C}(X_{n+1})) \geq 1 - \alpha$$

Where:

- $\mathbb{P}$ denotes the probability measure taken over the exchangeable joint distribution of images and taxonomic labels.
- X_{n+1} represents the unseen test observation (the incoming citizen-science photograph features).
- Y_{n+1} is the unobserved, true ground-truth taxonomic label for that test observation.
- $\hat{C}(X_{n+1})$ is the dynamic prediction set systematically generated by the conformal algorithm, containing zero, one, or multiple candidate taxa.
- α is the user-defined nominal significance level (error tolerance). For example, setting $\alpha = 0.05$ rigorously enforces a 95% coverage guarantee.

This theorem provides a remarkably robust distribution-free guarantee. This means that exactly $1 - \alpha$ of all future predictions will correctly contain the true species within the returned set, completely regardless of the underlying data distribution, lighting conditions, or base model architecture.

8.3 Memory Complexity of Hierarchical Vector Graphs

Retrieval-augmented generation systems frequently bottleneck on disk I/O when attempting to query massive vector databases stored on solid-state drives. In order to guarantee sub-millisecond retrieval latency during the Single-Stage Metadata-Filtered HNSW traversal, the entire Reference Exemplar Catalogue must clearly fit completely within the host system’s Random Access Memory (RAM).

In order to achieve this, the architecture aggressively bounds the catalogue size. Assuming the system scales to support a global baseline of 10,000 distinct taxa, and curates exactly 50

environmental crops per taxon, the vector index reaches a safe, deterministic upper bound of 500,000 nodes.

The theoretical memory lower bound $\mathcal{M}_{\text{index}}$ required to securely store the raw embedding vectors in memory is mathematically calculated as:

$$\mathcal{M}_{\text{index}} = N_{\text{nodes}} \times D_{\text{dim}} \times B_{\text{bytes}}$$

Where:

- $\mathcal{M}_{\text{index}}$ is the total theoretical memory footprint required to store the raw embedding vectors in RAM.
- N_{nodes} is the absolute upper bound of discrete biological exemplars within the retrieval catalogue (500,000).
- D_{dim} is the feature dimensionality of the chosen Vision Foundation Model (for example, 768 dimensions for a standard ViT-Base backbone).
- B_{bytes} is the byte-size of the numerical precision format (4 bytes for standard FP32).

Evaluating this expression logically yields:

$$\mathcal{M}_{\text{index}} = 500,000 \times 768 \times 4 \text{ bytes} = 1,536,000,000 \text{ bytes} \approx 1.53 \text{ GB}$$

By formally proving that the uncompressed vector payload requires approximately 1.53 GB, the system firmly guarantees that the entire HNSW graph (including navigational edge overhead) will safely consume less than 3 GB of host memory. This critically allows the index to remain resident in fast RAM concurrently alongside the FastAPI worker processes, thereby completely bypassing slow hard-drive reads and successfully preserving the 25 ms inference SLA.

Bibliography

- [1] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021.
- [2] Yin Cui, Menglin Jia, Tsung-Yi Lin, Yang Song, and Serge Belongie. Class-balanced loss based on effective number of samples. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9268–9277, 2019.
- [3] Pengzhen Ren, Yun Xiao, Xiaojun Chang, Po-Yao Huang, Zhihui Li, Brij B. Gupta, Xiaojiang Chen, and Xinwang Wang. A survey of deep active learning. *ACM Computing Surveys (CSUR)*, 54(9):1–40, 2021.
- [4] Samuel Stevens, Jiaman Wu, Matthew J. Thompson, Elizabeth G. Campolongo, Chan Hee Song, David E. Carlyn, Lifan Dong, Wasila M. Dahdul, Charles Stewart, Tanya Berger-Wolf, Wei-Lun Chao, and Yu Su. Bioclip: A vision foundation model for the tree of life. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19412–19424, 2024.
- [5] Grant Van Horn, Oisin Mac Aodha, Yang Song, Yin Cui, Chen Sun, Alex Shepard, Hartwig Adam, Pietro Perona, and Serge Belongie. The inaturalist species classification and detection dataset. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8769–8778, 2018.